



CSL303/MAL505 — Database Management Systems

Module II: Database Design

Lecture 14: EER & Mapping ER $\rightarrow$ Relations

Amit Kumar Dhar

IIT Bhilai

Recap & Goal

- Last time: entities, attributes, relationships, cardinalities, weak entities.
- Two jobs today:
 1. **Extended ER (EER)** — richer constructs: generalization/specialization, aggregation.
 2. The **mapping algorithm** — a mechanical recipe from ER/EER to a relational schema you can `CREATE TABLE`.

Agenda

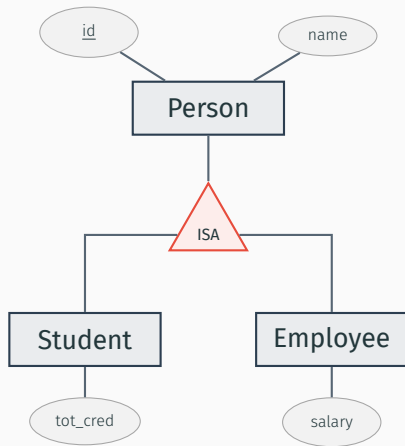
- Why ER isn't quite enough → EER
- Specialization & generalization (ISA)
- Attribute & disjointness constraints
- Completeness (total/partial) constraints
- Aggregation
- The 7-step ER→relational mapping algorithm
- Mapping each cardinality
- Mapping weak entities, multivalued attributes, ISA
- Full worked example

Why EER?

- Plain ER can't naturally express “a is a kind of b”.
- Real domains have hierarchies:
 - employee → instructor, secretary, technician.
 - account → savings, checking.
- Enhanced ER (EER) adds **generalization/specialization** and **aggregation** to capture these.

Specialization

- Start with a **superclass** entity set and identify **sub-groups** with extra attributes or relationships.
- **person specializes into student and employee.**
- Top-down: “a person is sometimes a student, sometimes an employee”.
- Drawn with an **ISA triangle** pointing at the superclass.



- The **bottom-up** dual: notice **car** and **truck** share attributes → factor out a common superclass **vehicle**.
- Specialization and generalization produce the **same diagram**; they differ only in the **direction of thinking**.
- Collectively: an **ISA hierarchy**.

Attribute Inheritance

- A subclass **inherits** all attributes of its superclass...
- ...**plus** its own specific attributes.
- **student** inherits **person.name**, **person.id**; adds **tot_cred**.
- It also inherits the superclass's **relationship** participations.
- The primary key is **always inherited** from the superclass.

Disjointness Constraint

- Can an entity belong to **more than one** subclass at once?
- **Disjoint (d)** — an entity is in **at most one** subclass.
 - an **account** is savings **or** checking, not both.
- **Overlapping (o)** — an entity may be in **several** subclasses.
 - a **person** may be both **student** and **employee**.

Completeness Constraint

- Must **every** superclass entity belong to **some** subclass?
- **Total specialization** (double line) — every superclass entity is in a subclass. (Every **vehicle** is a car or truck.)
- **Partial specialization** — some superclass entities are in **no** subclass. (A **person** who is neither student nor employee.)
- Independent of disjointness → four combinations.

Disjointness Constraint

Disjoint (d): At most one subclass.

Overlapping (o): Multiple subclasses allowed.

Completeness Constraint

Total (=): Every entity in subclass.

Partial (-): Some entities in no subclass.

Question

If **Employee** has subclasses **Salaried** and **Hourly**, and every employee must belong to exactly one of these two categories, which constraints apply to this specialization?

Question

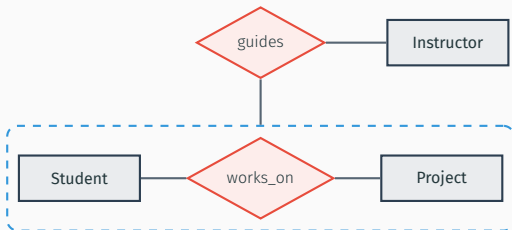
If **Employee** has subclasses **Salaried** and **Hourly**, and every employee must belong to exactly one of these two categories, which constraints apply to this specialization?

Answer

- **Disjointness: Disjoint (d)** — an employee cannot be both salaried and hourly.
- **Completeness: Total** — every employee must be either salaried or hourly.
- Constraint annotation: (d, total).

Aggregation

- Problem: a relationship can't directly participate in **another** relationship.
- *e.g.* (**student** works_on **project**) — and an **instructor** **guides** that whole working-on.
- **Aggregation** treats a relationship (plus its entities) as a **higher-level entity** that can itself have relationships.
- Drawn by boxing the relationship and connecting a new diamond to the box.



Quick Check — Aggregation

Question

Why is **Aggregation** preferred over a single 3-way (ternary) relationship between **Student**, **Project**, and **Instructor**?

Quick Check — Aggregation

Question

Why is **Aggregation** preferred over a single 3-way (ternary) relationship between **Student**, **Project**, and **Instructor**?

Answer

- A ternary relationship requires all three entities to participate **simultaneously** (a student, a project, AND an instructor together).
- Aggregation allows a student to work on a project **before** an instructor is assigned to guide them, and allows treating the **works_on** pairing as an independent object.

From Diagram to Tables: The Plan

- We now have the full modelling vocabulary. Time to **realize** it.
- The mapping is **mechanical** — 7 steps, applied in order.
- Output: a set of relation schemas with primary and foreign keys.
- Do it by hand a few times; then tools can do it, but you'll trust the result.



Step 1 — Strong Entity Sets

- Each strong entity set → **one relation** with all its **simple** attributes.
- Its primary key becomes the relation's primary key.

```
CREATE TABLE department (  
    dept_name TEXT PRIMARY KEY,  
    building  TEXT,  
    budget   NUMERIC  
);
```

- Composite attributes: **flatten** into their simple components (name → first_name, last_name).

Step 2 — Weak Entity Sets

- Weak entity → relation with its own attributes **plus** the owner's primary key as a **foreign key**.
- Primary key = owner's key + the weak entity's discriminator.

```
CREATE TABLE section (  
    course_id TEXT REFERENCES course(course_id), -- owner's key  
    sec_id     TEXT, semester TEXT, year INTEGER, -- discriminator  
    PRIMARY KEY (course_id, sec_id, semester, year)  
);
```

Step 3 — Multivalued Attributes

- A multivalued attribute → its **own** relation.
- Columns: the owning entity's primary key + one value per row.

```
CREATE TABLE instructor_phone (  
    inst_id TEXT REFERENCES instructor(id),  
    phone   TEXT,  
    PRIMARY KEY (inst_id, phone)  
);
```

- One instructor with three phones → three rows. This is the relational way to store a **set**.

Step 4 — 1:1 Relationships

- Two options; both correct:
 - **Merge** — put the relationship's attributes and one side's key into the other entity's table (good when participation is total on one side).
 - **Separate** — a table with either key as PK (and the other UNIQUE).
- Choose the side with **total participation** to hold the foreign key (fewer NULLs).

Step 5 — 1:N Relationships

- **No new table needed.** Put the “one” side’s primary key as a **foreign key** in the “many” side’s table.
- Relationship attributes also go on the **many** side.

```
-- advisor: one instructor advises many students
ALTER TABLE student ADD COLUMN advisor_id TEXT
    REFERENCES instructor(id);    -- FK on the "many" side
```

Step 6 — M:N Relationships

- Always a new table (a junction / associative table).
- Columns: the primary keys of **both** sides + any relationship attributes.
- Primary key = the **combination** of both foreign keys.

```
CREATE TABLE takes (  
    id          TEXT REFERENCES student(id),  
    course_id TEXT, sec_id TEXT, semester TEXT, year INTEGER,  
    grade       TEXT,                                -- relationship attribute  
    PRIMARY KEY (id, course_id, sec_id, semester, year)  
);
```

Step 7 — n-ary Relationships & Aggregation

- A relationship of degree $\geq 3 \rightarrow$ its own table with the PK of **each** participant + relationship attributes.
- Primary key = union of participants' keys (drop one if a cardinality allows).
- **Aggregation** maps like a relationship whose “entity” is the aggregated relationship's table — use that table's key as the foreign key.

Quick Check — 1:N vs M:N Mapping

Question

Suppose **Department** has a 1:N relationship **manages** with **Employee** (one department has many employees, each employee belongs to one department). How many relational tables are created for this relationship?

Quick Check — 1:N vs M:N Mapping

Question

Suppose **Department** has a 1:N relationship **manages** with **Employee** (one department has many employees, each employee belongs to one department). How many relational tables are created for this relationship?

Answer

- **0 additional tables!** Only 2 tables total (**Department** and **Employee**).
- The 1:N relationship does **not** require a junction table. We simply place **dept_id** as a foreign key inside the **Employee** table (the “many” side).

Mapping the ISA Hierarchy — 3 Methods

Method	Tables	Best when
A. Superclass + subclass tables	one per class; subclass FK→super	overlapping / partial
B. Only subclass tables	one per subclass, attrs duplicated	disjoint and total
C. Single table	one table + a type column, NULLs	few subclass-specific at

- Trade-off: number of joins vs number of NULLs vs enforcing completeness.

ISA Mapping — Method A (General)

```
CREATE TABLE person (  
    id    TEXT PRIMARY KEY,  
    name TEXT  
);  
CREATE TABLE student (  
    id      TEXT PRIMARY KEY REFERENCES person(id),  -- shared key  
    tot_cred INTEGER  
);  
CREATE TABLE employee (  
    id      TEXT PRIMARY KEY REFERENCES person(id),  
    salary  NUMERIC  
);
```

- Always works (any disjointness/completeness). Cost: a join to see the whole entity.

ISA Mapping — Method C (Single Table)

```
CREATE TABLE person (  
    id          TEXT PRIMARY KEY,  
    name        TEXT,  
    ptype       TEXT CHECK (ptype IN ('student', 'employee')),  
    tot_cred    INTEGER,    -- NULL for employees  
    salary      NUMERIC     -- NULL for students  
);
```

- One table, no joins — but many NULLs and weaker constraints. Fine when subclasses differ by only a column or two.

Choosing an ISA Method — Heuristics

- **Total + disjoint**, subclasses queried separately → **Method B**.
- **Overlapping** or **partial** → **Method A** (safest, most flexible).
- Subclasses barely differ, hierarchy shallow → **Method C**.
- When unsure, default to **A** — it never loses information.

Quick Check — ISA Hierarchy Mapping

Question

When mapping an **overlapping** ISA hierarchy where a **Person** can be both a **Student** and an **Employee**, why is Method B (only subclass tables) inappropriate?

Quick Check — ISA Hierarchy Mapping

Question

When mapping an **overlapping** ISA hierarchy where a **Person** can be both a **Student** and an **Employee**, why is Method B (only subclass tables) inappropriate?

Answer

- Method B duplicates superclass attributes (**name**, **address**) across separate subclass tables (**student** and **employee**).
- If a person is both a student and an employee, their basic info is stored twice, leading to data redundancy and update anomalies. **Method A** avoids this by storing shared attributes once in the **person** table.

Worked Example — The University (Setup)

Entities: `department`, `course` (weak owner of `section`), `instructor`, `student`.

Relationships:

- `course belongs_to department` (N:1)
- `section` is a weak entity of `course`
- `instructor/student belong_to department` (N:1)
- `advisor` (`instructor` : `student`, 1:N)
- `teaches` (`instructor` : `section`, M:N)
- `takes` (`student` : `section`, M:N, attr `grade`)
- `prereq` (`course` : `course`, M:N, recursive)

Worked Example — The Tables (1)

```
CREATE TABLE department (dept_name TEXT PRIMARY KEY, building TEXT, budget NUMERIC);
```

```
CREATE TABLE course (  
    course_id TEXT PRIMARY KEY, title TEXT, credits INTEGER,  
    dept_name TEXT REFERENCES department);           -- Step 5 (N:1)
```

```
CREATE TABLE instructor (  
    id TEXT PRIMARY KEY, name TEXT, salary NUMERIC,  
    dept_name TEXT REFERENCES department);           -- Step 5 (N:1)
```

Worked Example — The Tables (2)

```
CREATE TABLE student (  
    id TEXT PRIMARY KEY, name TEXT, tot_cred INTEGER,  
    dept_name TEXT REFERENCES department,  
    advisor_id TEXT REFERENCES instructor);           -- Step 5: advisor 1:N  
  
CREATE TABLE section (                               -- Step 2: weak entity  
    course_id TEXT REFERENCES course, sec_id TEXT,  
    semester TEXT, year INTEGER, building TEXT, room_no TEXT,  
    PRIMARY KEY (course_id, sec_id, semester, year));
```

Worked Example — The Tables (3)

```
CREATE TABLE teaches (                                -- Step 6: M:N
  id TEXT REFERENCES instructor,
  course_id TEXT, sec_id TEXT, semester TEXT, year INTEGER,
  PRIMARY KEY (id, course_id, sec_id, semester, year));

CREATE TABLE takes (                                  -- Step 6: M:N + attr
  id TEXT REFERENCES student,
  course_id TEXT, sec_id TEXT, semester TEXT, year INTEGER,
  grade TEXT,
  PRIMARY KEY (id, course_id, sec_id, semester, year));

CREATE TABLE prereq (                                -- Step 6: recursive M:N
  course_id TEXT REFERENCES course,
  prereq_id TEXT REFERENCES course,
  PRIMARY KEY (course_id, prereq_id));
```

- This is the `university.sql` schema you've been querying since Week 1.

Sanity-Checking the Mapping

- Every entity set has a table with a primary key. ✓
- Every M:N relationship became its own table. ✓
- Every 1:N put an FK on the many side (no junction table). ✓
- No multivalued or composite attributes remain as single columns. ✓
- Every FK references an existing primary key. ✓

Losing Information: Watch-outs

- Merging a 1:1 into one side is fine — but the **wrong** side forces NULLs.
- Collapsing an M:N onto one side **loses** rows — never do it.
- Dropping a weak entity's owner FK breaks identification.
- Over-eager ISA Method C can't enforce “every person is student or employee”.
- Rule of thumb: **preserve every constraint the diagram promised.**

From Logical to Physical (Preview)

- The mapping gives a **logical** schema — correct, but not yet tuned.
- **Week 6** checks it for redundancy (normalization).
- **Weeks 7+** decide the **physical** layout: files, pages, indexes.
- A clean ER→relational mapping usually lands close to **3NF** already.

Mapping a Ternary Relationship

- supplier supplies part to project (degree 3, attribute **qty**):

```
CREATE TABLE supplies (  
  supplier_id TEXT REFERENCES supplier,  
  part_id     TEXT REFERENCES part,  
  project_id  TEXT REFERENCES project,  
  qty         INTEGER,  
  PRIMARY KEY (supplier_id, part_id, project_id));
```

- One table, the PK of **every** participant; drop a participant from the key only if a cardinality constraint makes it functionally determined.

When to Merge vs Keep Separate

- A 1:1 with **total participation on one side** → **merge** into that side (no extra table, no NULLs).
- A 1:1 with **partial** participation both sides → keep **separate** (merging forces NULLs).
- A 1:N → never a separate table; FK on the many side.
- Rule: **minimize NULLs and tables** without losing a constraint.

Surrogate Keys During Mapping

- Weak entities and M:N tables get **composite** keys — fine, but they cascade into every referencing table.
- Many designers add a **surrogate** PK (`section_id SERIAL`) and keep the natural composite as a **UNIQUE** constraint.
- Trade-off: simpler FKs elsewhere vs an extra, meaningless column.
- We keep **natural composite keys** in `university.sql` for teaching clarity.

A Common Mapping Mistake

```
-- WRONG: an M:N 'takes' collapsed onto student loses enrolments  
ALTER TABLE student ADD COLUMN course_id TEXT; -- one course per student?!
```

- Putting an FK on **one** side of an **M:N** relationship can store only **one** partner
→ data loss.
- **Every** M:N needs its **own** table. If you're tempted to add a single FK, re-check the cardinality.

Verifying with Sample Data

- After mapping, **insert a few rows** and run the queries the design must answer.
- Does a student with two courses produce two **takes** rows? ✓
- Can you find sections with no instructor via a **LEFT JOIN**? ✓
- A schema that can't answer its intended queries is **wrong**, however pretty the diagram.

Summary — Key Takeaways

- **EER** adds ISA hierarchies (with disjointness & completeness) and aggregation.
- ER→relational is a **7-step mechanical algorithm**.
- 1:N → FK on the many side; **M:N → its own table**; weak entity → owner's key + discriminator.
- **Multivalued** attributes and **ISA** hierarchies each get their own tables (with method choices).

Exercises

1. Map your Lecture-13 **library** ER diagram to relations.
2. Map an ISA hierarchy **account** $\rightarrow$ {**savings**, **checking**} three ways; compare.
3. Given a 1:1 **employee manages department**, write both merged and separate schemas. Which minimizes NULLs?
4. Show why an M:N relationship cannot be mapped with just a foreign key.

Quick Reference — Mapping Rules

ER construct	Relational result
Strong entity	table, its PK
Weak entity	table, owner FK + discriminator as PK
1:1	FK on either side (prefer total side)
1:N	FK on the N side
M:N	new table , both PKs combined
Multivalued attr	new table (owner PK + value)
ISA	Methods A / B / C

Before Next Lecture

- Complete the library mapping; bring it to lab.
- Optional reading: Silberschatz §6.6–§6.9 (EER & reduction to schemas).

Next time — Lecture 15: an **end-to-end design case study**, alternative designs, and the **pitfalls** that make schemas go wrong.

Questions?